

Code Walkthrough: Mechanics of the Q-Learning Agent

This section breaks down the implementation of the Q-Learning algorithm within the Stochastic Gridworld environment, detailing how the agent represents the world, transitions between states, and updates its knowledge over time.

1. Environment Representation (The MDP)

The environment, defined in the `StochasticGridworld` class, is a discrete representation of a Markov Decision Process (MDP).

States (S)

The state space consists of every valid (x, y) coordinate on a 5×5 grid.

- **Agent Representation:** The agent tracks its current position as a simple tuple:
`self.current_state = (x, y)`.
- **Total States:** 25 distinct states.
- **Special States:** The Start state is $(0, 0)$. The Goal state is $(4, 4)$. The Pits are located at $(1, 1)$, $(2, 1)$, and $(3, 1)$.

Actions (A)

At any given state, the agent has four possible discrete actions available to it:

- 0: Up
- 1: Right
- 2: Down
- 3: Left

Rewards (R)

The reward function is baked into the environment's `step()` method. The agent receives immediate feedback after every action:

- **Goal:** $+10.0$
- **Pit:** -10.0
- **Standard Step:** -0.1 (This small penalty encourages the agent to find the shortest safe path, rather than wandering endlessly).

2. The Transition Matrix (P) and Stochasticity

In a deterministic environment, taking action a in state s guarantees moving to the intended next state s' . However, our environment incorporates a `slip_prob` (e.g., 0.3 or 30%). This means the transition is stochastic, forming the core Markov property of the system.

While the code does not explicitly build a massive 25×25 transition matrix, the logic in the `step(action)` function mathematically functions exactly as one.

When the agent attempts to move (e.g., Action = 0 [Up]):

- **Intended Move ($1 - \text{slip_prob}$):** With a 70% chance, the agent moves Up.
- **Slip Left ($\text{slip_prob}/2$):** With a 15% chance, the agent slips and moves Left.
- **Slip Right ($\text{slip_prob}/2$):** With a 15% chance, the agent slips and moves Right.

If the agent is at state $s = (1, 0)$ and chooses action $a = 0$ (Up), the implicit transition probabilities $P(s' \mid s, a)$ are:

- $P((1, 1) \mid (1, 0), \text{Up}) = 0.70$ (Moves into the pit!)
- $P((0, 0) \mid (1, 0), \text{Up}) = 0.15$ (Slips Left)
- $P((2, 0) \mid (1, 0), \text{Up}) = 0.15$ (Slips Right)

The agent does not know these probabilities beforehand; it must discover them through interaction.

3. The Learning Loop: Updating the Q-Table

The "brain" of the agent is the Q-table, initialized as a 3D NumPy array of zeros: `np.zeros((5, 5, 4))`. This table stores the expected cumulative return (Q-value) for taking a specific action in a specific state.

The ϵ -Greedy Strategy (Exploration vs. Exploitation)

In the `choose_action()` method, the agent decides its next move.

- It generates a random number. If the number is less than ϵ (epsilon), it selects a random action to explore the map.
- Otherwise, it exploits its current knowledge by looking at the Q-table for its current state and choosing the action with the highest value (`np.argmax(self.q_table[x, y])`).
- Epsilon decays over time, meaning the agent explores wildly at first, but slowly shifts to pure exploitation as its Q-table becomes more accurate.

The Bellman Update

After every step, the agent receives a tuple: (`state`, `action`, `reward`, `next_state`). It uses this experience to update its Q-table via the Temporal Difference (TD) learning formula (the Bellman equation):

```
1 best_next_q = np.max(self.q_table[nx, ny])
2 td_target = reward + self.gamma * best_next_q
3 td_error = td_target - self.q_table[x, y, action]
4 self.q_table[x, y, action] += self.alpha * td_error
```

1. **Look Ahead (`best_next_q`):** The agent looks at the state it just landed in (`nx`, `ny`) and finds the maximum possible Q-value available from that state. This is its estimate of future rewards.
2. **Calculate Target (`td_target`):** It adds the immediate `reward` it just received to the discounted future rewards ($\gamma \times \text{best_next_q}$). This forms the "target" value.
3. **Calculate Error (`td_error`):** It compares this new target value to its old estimate (`self.q_table[x, y, action]`).
4. **Update Table:** It adjusts its old estimate in the direction of the error, scaled by the learning rate (α).

Through thousands of episodes, the negative -10 rewards from falling into the pits propagate backward through the grid via this equation, lowering the Q-values of actions that lead near the pits, ultimately teaching the agent to avoid those states entirely.